

Project Spec — CatsCarsCoins

Package: `com.icodeforyou.catscarscoins` **Purpose:** A demonstration Android app on the *latest but stable* Jetpack/Compose stack that doubles as a reference blueprint for Clean Architecture, Clean Code, and SOLID.

Signatures and schema code shown are the contracts to build to. Version numbers are pinned in `libs.versions.toml`.

1. Goals / Non-Goals

Goals

- Exercise the current Jetpack + Compose stack end to end.
- Be a copyable blueprint where each layer demonstrates Clean Architecture / Clean Code / SOLID.
- Prove one generic list/detail component.
- Drive three features off three different source shapes.
- Build feature-at-a-time, simplest to most complex, test-first.

Non-Goals

- Not production-hardened.
- The SQL query tool is an intentional foot-gun.
- No auth, no analytics, no offline-first guarantee.
- No undo / no confirm on destructive actions (explicit decision).

2. Build Targets

Setting	Value	Where defined
Kotlin / KGP	2.2.10	<code>libs.versions.toml</code> <code>[versions] kotlin</code> ; applied by AGP built-in Kotlin
JVM target	<code>JavaVersion.VERSION_17</code>	<code>app/build.gradle.kts</code> <code>-> compileOptions + kotlin { jvmToolchain(17) }</code>
AGP	9.2.1	<code>libs.versions.toml</code> <code>[versions] agp</code> ; <code>settings.gradle.kts</code> <code>pluginManagement</code>
Gradle	9.4.1	<code>gradle/wrapper/gradle-wrapper.properties</code> <code>distributionUrl</code>
<code>compileSdk</code>	37	<code>app/build.gradle.kts</code> <code>-> android { compileSdk { version = release(37) } }</code>
<code>minSdk</code>	31	<code>app/build.gradle.kts</code> <code>-> defaultConfig { minSdk = 31 }</code>
<code>targetSdk</code>	36	<code>app/build.gradle.kts</code> <code>-> defaultConfig { targetSdk = 36 }</code>
Compose BOM	2026.02.01	<code>app/build.gradle.kts</code> <code>platform(...)</code> ; ref in <code>libs.versions.toml</code>
Room schema export	on	<code>app/build.gradle.kts</code> <code>ksp { arg("room.schemaLocation", ...) }</code>
All dependency versions	—	<code>gradle/libs.versions.toml</code> (single source of truth)

3. Architecture

- **Style:** Clean Architecture per feature; **single Gradle module** (`:app`) organized package-by-feature. No library modules — the SQL popover and query tool live in the app code.
- **Split-readiness:** the package layout is chosen so a later split into `:core:*` / `:feature:*` modules is a move-folders operation, not a rewrite.

- **Dependency rule (inward-only):** `ui -> domain <- data`. The domain layer is pure Kotlin — no Android, Room, Retrofit, or Compose. Framework types live only in `data` and `ui`.
- **Boundaries:** every cross-layer boundary is an interface owned by the inner layer (DIP). `domain` declares `CatRepository`; `data` implements it.
- **Per-feature anatomy:** `data/{remote, local, mapper, repository}`, `domain/{model, repository, usecase}`, `ui/{list, detail, component}`, `nav/, di/`.
- **Cross-feature needs:** expressed as small interfaces (e.g. `RefreshableTable`, `ClearableTable`) that each feature implements and the consumer depends on abstractly — no feature reaches into another's internals.
- **State & threading:** unidirectional data flow; ViewModels expose immutable `StateFlow<UiState>`; UI collects with `collectAsStateWithLifecycle()`; work runs on an injected `DispatcherProvider` (never hard-coded `Dispatchers.IO`). No `LiveData`.
- **DI:** Koin runtime DSL; one module per feature; `di/AppModules.kt` aggregates; `MyApp` calls `startKoin`.
- **Navigation:** `Nav3` (`NavDisplay` + developer-owned back stack) for screens, wrapped by `NavigationSuiteScaffold` for the About/Database/Prefs chrome.
- **Persistence split:** structured/queryable data -> Room; scalar settings -> Preferences `DataStore`. No overlap.

4. Manifest & Permissions

Minimal by design. Only network access needs a permission.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

    <application
        android:name=".MyApp"
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:theme="@style/Theme.CatsCarsCoins">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.CatsCarsCoins">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Concern	Permission / entry	Needed?	Why
HTTP calls	INTERNET	Yes	Required for any network I/O.
Connectivity checks	ACCESS_NETWORK_STATE	Optional	Only if polling/refresh is gated on connectivity.
Cleartext HTTP	usesCleartextTraffic	No	All three APIs are HTTPS.
Toasts (Notifier)	POST_NOTIFICATIONS	No	In-app Compose overlay, not a system notification.
Database	storage permission	No	Room lives in app-internal storage.
Haptics	VIBRATE	No	Compose <code>HapticFeedback.performHapticFeedback()</code> needs no permission.
Reset sound	audio permission	No	<code>SoundPool</code> playback of a bundled

			<code>res/raw</code> asset needs none.
Splash	—	No	Platform SplashScreen via theme + <code>installSplashScreen()</code> .

Config note: do **not** add `uiMode` to any `android:configChanges` — letting the system deliver the `uiMode` change is what makes `theme = system` follow a live device dark/light switch.

5. Technology Stack

Concern	Choice	Rationale
Navigation (screens)	Navigation 3	Compose-first, developer-owned back stack, adaptive multi-pane via Scenes.
Navigation (chrome)	<code>NavigationSuiteScaffold</code>	Auto NavigationBar (compact) / NavigationRail (medium) by window size.
Adaptive list/detail	<code>material3.adaptive:adaptive-navigation3</code>	List-detail SceneStrategy.
DI	Koin — runtime DSL	Lightweight, no codegen.
Persistence (domain)	Room + <code>@Fts4</code> (KSP)	One <code>CatsCarsDatabase</code> ; FTS, no hand-written triggers.
Persistence (prefs)	Preferences DataStore	Flow-based scalar prefs.
Networking	Retrofit + OkHttp	Logging interceptor; kotlinx converter.
(De)serialization	<code>kotlinx.serialization</code>	Kotlin-native; also serializes Nav3 <code>@Serializable</code> <code>NavKey</code> s.
Images/cache/SVG	Coil 3 (<code>coil-compose</code> , <code>coil-svg</code> , <code>coil-network-okhttp</code>)	One lib for caching + SVG.
UI	Compose + Material3	Single-Activity; swipe/refresh/segmented/slider/switch primitives.
Notify	Custom Notifier host	Arbitrary composable payload; dismissible.
Audio	<code>SoundPool</code> + OGG	Reset SFX.
SQL popover + query tool	In-app composables (in <code>feature/database</code>)	Not a separate library.
Test	JUnit4, coroutines-test, Turbine, MockK, Room-testing, Compose-ui-test, Kover	See Build Plan.

6. Data Sources

Source	Endpoint	Auth
The Cat API	<code>thecatapi.com</code> (breeds + images)	API key via <code>local.properties</code> -> <code>BuildConfig</code> (gitignored).
NHTSA VPIC	<code>vpic.nhtsa.dot.gov/api</code>	None.
Coinbase spot	<code>GET api.coinbase.com/v2/prices/BTC-USD/spot</code>	None (public, ~10 req/s).

7. Database Schema (CatsCarsDatabase , Room)

One Room database, version 1, `exportSchema = true`. **Local primary keys are uniformly named `id`** (wire IDs like `Mfr_ID` mapped to `id` at the data boundary), so the viewer's delete path is uniform: `DELETE FROM <table> WHERE id = ?`. The `id` **column type** varies per table (String / Int / Long); only the name is uniform. Money is stored as **integer minor units** (`amount_cents`), never a floating-point `Double`.

7.1 Coins

```
@Entity(tableName = "coins")
data class CoinEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0L,
    @ColumnInfo(name = "amount_cents") val amountCents: Long,    // 63142.55 USD -> 6_314_255
    @ColumnInfo(name = "recorded_at") val recordedAt: Long,      // epoch millis; view-only
)
```

```
CREATE TABLE coins (
    id            INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    amount_cents INTEGER NOT NULL,
    recorded_at   INTEGER NOT NULL
);
```

```
@Insert suspend fun insert(row: CoinEntity): Long
```

```
@Query("DELETE FROM coins WHERE id NOT IN (SELECT id FROM coins ORDER BY id DESC LIMIT :limit)")
suspend fun trimToLimit(limit: Int)           // default 500; called after each insert
```

7.2 Cats

```
@Entity(tableName = "cats")
data class CatEntity(
    @PrimaryKey val id: String,                // Cat API image id
    @ColumnInfo(name = "image_url") val imageUrl: String,
    @ColumnInfo(name = "breed_name") val breedName: String,
    @ColumnInfo(name = "origin") val origin: String,
    @ColumnInfo(name = "temperament") val temperament: String,
    @ColumnInfo(name = "description") val description: String,
    @ColumnInfo(name = "life_span") val lifeSpan: String,
)

@Fts4(contentEntity = CatEntity::class)
@Entity(tableName = "cats_fts")
data class CatFtsEntity(
    @ColumnInfo(name = "breed_name") val breedName: String,
    @ColumnInfo(name = "temperament") val temperament: String,
    @ColumnInfo(name = "description") val description: String,
)
```

```
CREATE TABLE cats (
    id            TEXT PRIMARY KEY NOT NULL,
    image_url     TEXT NOT NULL, breed_name TEXT NOT NULL, origin TEXT NOT NULL,
    temperament   TEXT NOT NULL, description TEXT NOT NULL, life_span TEXT NOT NULL
);
CREATE VIRTUAL TABLE cats_fts USING fts4(content="cats", breed_name, temperament, description);
```

7.3 Cars — manufacturers (+ child makes)

```
@Entity(tableName = "manufacturers")
data class ManufacturerEntity(
    @PrimaryKey val id: Int,                    // <- Mfr_ID
    @ColumnInfo(name = "name") val name: String, // <- Mfr_Name
    @ColumnInfo(name = "country") val country: String? = null, // detail-only, nullable
    @ColumnInfo(name = "detail_loaded") val detailLoaded: Boolean = false,
)

@Entity(
    tableName = "makes",
    foreignKeys = [ForeignKey(entity = ManufacturerEntity::class,
        parentColumns = ["id"], childColumns = ["mfr_id"], onDelete = ForeignKey.CASCADE)],
    indices = [Index("mfr_id")],
)
```

```

data class MakeEntity(
    @PrimaryKey val id: Int,                // <- Make_ID
    @ColumnInfo(name = "make_name") val makeName: String, // <- Make_Name
    @ColumnInfo(name = "mfr_id") val mfrId: Int,          // FK -> manufacturers.id
)

@Fts4(contentEntity = ManufacturerEntity::class)
@Entity(tableName = "manufacturers_fts")
data class ManufacturerFtsEntity(@ColumnInfo(name = "name") val name: String)

@Fts4(contentEntity = MakeEntity::class)
@Entity(tableName = "makes_fts")
data class MakeFtsEntity(@ColumnInfo(name = "make_name") val makeName: String)

```

```

CREATE TABLE manufacturers (
    id INTEGER PRIMARY KEY NOT NULL, name TEXT NOT NULL,
    country TEXT, detail_loaded INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE makes (
    id INTEGER PRIMARY KEY NOT NULL, make_name TEXT NOT NULL, mfr_id INTEGER NOT NULL,
    FOREIGN KEY (mfr_id) REFERENCES manufacturers(id) ON DELETE CASCADE
);
CREATE VIRTUAL TABLE manufacturers_fts USING fts4(content="manufacturers", name);
CREATE VIRTUAL TABLE makes_fts USING fts4(content="makes", make_name);

```

- VPIC search unions `manufacturers_fts + makes_fts` (`mfr_name + make_name`).
- `makes / makes_fts` populate **lazily** as a manufacturer's makes are fetched on detail.
- The Database Viewer's **CARS segment shows** `manufacturers`; `makes` is a child table, browsed via the feature detail, not a viewer segment.

7.4 Database class

```

@Database(
    entities = [
        CatEntity::class, CatFtsEntity::class,
        ManufacturerEntity::class, ManufacturerFtsEntity::class,
        MakeEntity::class, MakeFtsEntity::class,
        CoinEntity::class,
    ],
    version = 1, exportSchema = true,
)
abstract class CatsCarsDatabase : RoomDatabase() {
    abstract fun catDao(): CatDao
    abstract fun manufacturerDao(): ManufacturerDao
    abstract fun makeDao(): MakeDao
    abstract fun coinDao(): CoinDao
    abstract fun rawQueryDao(): RawQueryDao // @RawQuery for the SQL reader + viewer
}

```

- **Migrations:** demo policy `fallbackToDestructiveMigration()`.
- **Dynamic-table ops** (viewer delete / count / sorted read) use `@RawQuery` with the table name from the trusted `TableRegistry` allow-list — never user text — so interpolating the table name is safe.

8. Screens

Screen	Role
Splash	Platform splash suppressed to a black window (values-v31 + 1x1 black icon vector); Compose splash holds ~2400 ms, then -> Main.
Main	Big CATS / CARS / COINS header + three feature buttons; nav chrome hosts About / Database / Prefs.
Cats	Cat API list/detail with custom cards.
Cars	VPIC manufacturer list -> detail -> nested Makes list.
Coins	Live BTC-USD ticker.
About	Placeholder.
Database Viewer	Raw table browser + query/refresh/search/count + sort + swipe-delete.
Preferences	theme / polling / pause / haptics + Reset App.

9. Main Screen

- Big **CATS / CARS / COINS** header in the safe area (safeDrawingPadding()).
- Spacing, then three feature buttons in that order.
- Tapping a button opens that **list/detail feature** (not a raw table).
- No gear, no FAB. About / Database / Prefs is the nav chrome (Section 10).

10. Navigation Chrome

- Destinations: **About, Database, Prefs** — hosted in a `NavigationSuiteScaffold`.
- Form by window size: **NavigationBar** on compact (phone portrait); **NavigationRail** on medium (landscape / small tablet); chosen automatically from the size class.
- Items are icon + text; Nav3's `NavDisplay` sits inside.
- Routing: About -> placeholder; Database -> Database Viewer; Prefs -> Preferences.

11. Generic List/Detail Component (`core/ui/list`)

The reusable list. A feature supplies the data and the renderers; the component owns interaction and adaptive behavior.

```
SwipeableRefreshableList<T>(  
    items: List<T>, key: (T) -> Any, isRefreshing: Boolean,  
    onRefresh: () -> Unit, onDismiss: (T) -> Unit, onItemClick: (T) -> Unit,  
    itemContent: @Composable (T) -> Unit  
)
```

- `onDismiss` is behavior-agnostic — feature screens pass remove-from-view (restored on refresh); the Database Viewer passes a permanent delete. The component never knows about persistence.
- Interactions: right-to-left swipe -> `onDismiss`; pull-to-refresh -> `onRefresh`; tap -> `onItemClick`.

Adaptive (Nav3 list-detail `SceneStrategy`):

Form factor	Layout
Phone portrait	list <-> detail, with back
Phone landscape	list + detail/placeholder side by side
Tablet	list-detail together

12. Feature: Cats

- Cat API list/detail; Room-cached; Coil images.
- **FTS** over breed_name + temperament + description.
- Uses the generic list with a custom `CatCard`.

- Pull-to-refresh restores swipe-removed items.

13. Feature: Cars (VPIC)

Manufacturer trio; one generic envelope `VpicResponse<T>` (Count / Message / SearchCriteria / Results); `@SerializedName` at the wire, idiomatic Kotlin properties; `ignoreUnknownKeys = true`; DTO defaults = `null` / `emptyList()`.

Endpoint	Decodes as	Role
<code>GetAllManufacturers</code>	<code>VpicResponse<ManufacturerSummary></code>	entry list
<code>GetManufacturerDetails(mfrId)</code>	<code>VpicResponse<ManufacturerDetail></code>	<code>detail = results.first()</code>
<code>GetMakeForManufacturer(mfrId)</code>	<code>VpicResponse<ManufacturerMake></code>	nested Makes list

- `VehicleType` shared (nullable `gvwrFrom/gvwrTo`).
- Load limit `manufacturerLoadLimit: Int? = 500` (null = all) + sync spinner.
- FTS over `mfr_name` + `make_name` (make_name lazy).

14. Feature: Coins (ticker + persistence)

- Coinbase BTC-USD spot; poll interval read live from `polling` (5-30 s) -> `Flow` -> stream-adjacent dedup -> `StateFlow`.
- **Dedup** is consecutive-only vs the previous emission: `distinctUntilChanged { prev, next -> prev.amountCents == next.amountCents }`. Not `SELECT DISTINCT`, not a table lookup — two equal amounts that are non-adjacent are both kept.
- On a genuine change: append a row (then `trimToLimit(500)`) **and** raise a 1250 ms toast — both hang off the same emission.
- `pause = true` freezes the last value; changing the interval restarts the `Flow` in place.

15. Notifier (dismissible toast)

- One host mounted at the **root, above `NavDisplay`**; any screen can raise a payload.
- Coroutine-backed; exposes the current payload as state.
- Payload is an arbitrary `@Composable`.
- Shows for **up to** its duration, then auto-clears.
- **Dismiss early:** whole-surface tap OR a small `[X]` (top-right). Dismiss lives in the **host**, so every toast gets it; the payload stays a dumb visual.
- **Replace-on-collision** (latest wins).
- Consumers: coin change (**1250 ms**); Database Viewer “Read Only” (**750 ms**).

```
Notifier.show(durationMs: Long, content: @Composable () -> Unit)
```

16. Preferences

Preferences `DataStore`.

Key	Type	Values	Widget
<code>theme</code>	enum	dark / light / system	<code>SegmentedButton</code>
<code>polling</code>	Int	5-30 s, integer steps	<code>Slider</code> + label
<code>pause</code>	Bool	true / false	<code>Switch</code>
<code>haptics</code>	Bool	true / false (default ON)	<code>Switch</code>

Reset App — one press, no confirm. On tap, in order:

1. Wipe all tables (each feature repository clears its own).

- 2. `resetToDefaults()` -> `theme=dark`, `haptics=on`, `polling=5`, `pause=true`.
- 3. Play `db_wipe.ogg`.

Design:

- `ResetAppUseCase` **delegates** clearing to each feature repository — no central schema knowledge.
- Audio is a separate `ResetEffect` seam (OCP): a `SoundResetEffect` impl backed by `SoundEffectPlayer` (`SoundPool`, `R.raw.db_wipe`). The use case stays pure.

17. Buttons (haptics + font size)

Every `AppButton` takes two **required** params — no defaults. The button just reads and uses them; it has no knowledge of preferences or the design system.

```
AppButton(  
    text: String,  
    onClick: () -> Unit,  
    useHaptics: Boolean,      // caller passes hapticsEnabled()  
    fontSize: Float,         // caller passes buttonFontSize()  
)
```

Param	Type	Accessor (global)	Notes
useHaptics	Boolean	<code>hapticsEnabled()</code>	Buzz-at-tap value; read in <code>onClick</code> .
fontSize	Float	<code>buttonFontSize()</code>	Current global button text size.

- Both accessors are **plain top-level synchronous** functions read by the caller at the call site. The button receives plain values.
- A button never buzzes on draw: haptics fire in `onClick`, and recomposition does not invoke `onClick`.
- The **only** flow-observer is the Preferences haptics `Switch` (it *draws* the setting, so it observes the `DataStore` flow); its own buzz still uses `hapticsEnabled()` at tap time.
- `SqlQueryEditor`'s `useHaptics` param is wired from the same accessor.

18. Theme

- Two explicit palettes: `lightColorScheme()` / `darkColorScheme()` in `designsystem/theme/Color.kt`.
- Dynamic color **off** (deterministic blueprint).
- `system` resolves via `isSystemInDarkTheme()` (reads `LocalConfiguration`).
- A device dark/light flip delivers a config change -> Compose recomposes -> theme follows **live** (manifest must not suppress `uiMode`).
- Flow: `DataStore` -> `StateFlow` -> `MaterialTheme`; the tree recomposes on change. No manual refresh.

19. Database Viewer

Reached via the **Database** nav destination. **No FAB**.

Selector: single-select `SegmentedButton` — CATS / CARS / COINS (segments, not radio; an in-content view switch).

Control box (2x2), beside the selector:

```
+-----+-----+  
| Query (icon) | Refresh (icon) |  
+-----+-----+  
| Search (icon) | Count (#)      |  
+-----+-----+
```

Control	Action
Query	Opens the SQL popover. Guards (caller-side): trim -> first ; -chunk -> require case-insensitive start SELECT -> @RawQuery -> grid. Result is read-only.
Refresh	Cats/Cars: re-query backend, rebuild table (plain, deletable view). Coins: cancel in-flight poll wait, poll now, append iff amount changed, reset next poll; does not touch pause.
Search	FTS box. Disabled for COINS (numeric columns).
Count	Static; live row count for the selected table.

Table = `List<RecordComposable>` via `SchemalessGrid`:

- Swipe right-to-left **deletes from the DB** — permanent, no undo/confirm. `DELETE FROM <table> WHERE id = ?`. Live **only on plain table views**, never on query results.
- **Frozen header row** — column names stationary while records scroll. **Each header cell is a button; its tap drives the tri-state sort.**
- **Horizontal scroll** if wider than the screen; no column wrap. Header and rows share one horizontal scroll state.
- **Tri-state sort** (SQL `ORDER BY`): tap -> ^ ASC; tap same -> v DESC; tap third -> clear, natural (`id`) order; tap a different column -> reset to ASC.

Query results are read-only (decision A). A `SELECT` is a *view*, not a table (`JOIN`s duplicate `id`; aggregates/expressions have none; a projection may omit `id`), so none is safely deletable. Cues:

- Persistent **12 pt “Read Only”** label, top-right of the header.
- A **750 ms “Read Only” toast** when a result lands.
- Refresh (or re-selecting the segment) restores the plain, deletable table.

20. SchemalessGrid (core/ui/grid)

- One read-only base: renders a `Cursor`’s columns/rows with a **frozen header** and a **shared horizontal scroll**.
- Header cells are buttons that emit a column-tap.
- Serves both the SQL result and the viewer’s table (the result ignores the tap; the table routes it to sort).
- The live table view = this grid **composed with** swipe-delete (via the list component) **and** the sortable header — composition, not a flag-laden component.

21. Coding Standards

- SOLID / Clean Code lead every decision; principle-settled choices are not re-litigated.
- No magic values — `const val` in `UPPER_SNAKE_CASE` (top-level private `const val` or companion object).
- Domain stays framework-free.
- Complete files, one at a time.

22. Decision Log

Area	Decision
Package / order	<code>com.icodeforyou.catscarscoins</code> ; display + button order CATS / CARS / COINS.
Toolchain	AGP 9.2.1, Kotlin 2.2.10, Gradle 9.4.1, Compose BOM 2026.02.01, <code>compileSdk 37</code> , <code>minSdk 31</code> , <code>targetSdk 36</code> .
Modules	Single <code>:app</code> only (package-by-feature). Query tool + SQL popover are in-app code, not a library.
DI	Koin, runtime DSL.
Navigation	Nav3 for screens; <code>NavigationSuiteScaffold</code> (bar/rail) for chrome.
Persistence	One <code>CatsCarsDatabase</code> (Room + <code>@Fts4</code> , KSP); Preferences <code>DataStore</code> for prefs.
Manifest	Only <code>INTERNET</code> (+ optional <code>ACCESS_NETWORK_STATE</code>); no

	notification/vibrate/storage/cleartext; keep <code>uiMode</code> out of <code>configChanges</code> .
PK convention	Local PKs uniformly named <code>id</code> (wire IDs mapped in); enables uniform <code>DELETE ... WHERE id = ?</code> .
Money type	<code>amount_cents: Long</code> (integer minor units); never <code>Double</code> .
Coins — dedup	Consecutive-only on <code>amountCents</code> via <code>distinctUntilChanged</code> .
Coins — table	<code>{ id, amount_cents, recorded_at }</code> ; FIFO cap 500 via <code>trimToLimit</code> .
Cats FTS	<code>breed_name</code> + <code>temperament</code> + <code>description</code> (content-linked FTS4).
Cars / VPIC	Manufacturer trio + <code>VpicResponse<T></code> ; load limit <code>Int? = 500</code> ; FTS <code>mfr_name</code> + <code>make_name</code> (make lazy); viewer CARS segment = <code>manufacturers</code> .
Toast	Dismissible (tap or <code>[X]</code>); coin 1250 ms, read-only 750 ms; replace-on-collision.
Preferences	theme, polling (5–30 int), pause, haptics (default ON).
Reset App	One press, no confirm: wipe + defaults + <code>db_wipe.ogg</code> (SoundPool/OGG).
Buttons	Required <code>useHaptics: Boolean</code> and <code>fontSize: Float</code> , no defaults; caller passes <code>hapticsEnabled()</code> / <code>buttonFontSize()</code> .
Theme	Two palettes; dynamic off; <code>system</code> reactive via <code>isSystemInDarkTheme()</code> .
Database Viewer	Segments; 2x2 controls; tri-state <code>ORDER BY</code> via header buttons; <code>id</code> -keyed swipe-delete on plain views only.
Query results	Read-only always; 12 pt label + 750 ms toast; safe via <code>TableRegistry</code> allow-list.
Shared UI	<code>SchemalessGrid</code> shared by query + table; generic list <code>onDismiss</code> injected.
Destructive UX / FABs	No undo, no confirm — anywhere. No FABs — anywhere.

23. Full Folder Layout

One file per line.

```

catscarscoins/
├─ settings.gradle.kts
├─ build.gradle.kts
├─ gradle.properties
├─ local.properties
├─ gradle/
├─ └─ libs.versions.toml
├─ app/
│   └─ build.gradle.kts
│       └─ src/
│           └─ main/
│               ├── AndroidManifest.xml
│               ├── kotlin/com/icodeforyou/catscarscoins/
│               │   ├── MyApp.kt
│               │   ├── MainActivity.kt
│               │   ├── di/
│               │   │   └─ AppModules.kt
│               │   ├── navigation/
│               │   │   ├── NavKeys.kt
│               │   │   ├── AppNavDisplay.kt
│               │   │   └─ AppNavSuite.kt
│               │   ├── core/
│               │   │   ├── common/
│               │   │   │   ├── Result.kt
│               │   │   │   ├── DispatcherProvider.kt
│               │   │   │   └─ Constants.kt
│               │   │   └─ designsystem/
│               │   │       ├── theme/
│               │   │       │   ├── Theme.kt
│               │   │       │   └─ Color.kt

```

```
├─── Type.kt
│   ├── Shape.kt
│   └── Spacing.kt
├── component/
│   ├── AppTopBar.kt
│   └── AppButton.kt
├── ui/
│   ├── list/
│   │   └── SwipeableRefreshableList.kt
│   ├── adaptive/
│   │   └── ListDetailScaffold.kt
│   ├── grid/
│   │   └── SchemalessGrid.kt
│   ├── state/
│   │   └── UiState.kt
│   ├── haptics/
│   │   └── Haptics.kt
│   └── notifier/
│       ├── Notifier.kt
│       ├── NotifierHost.kt
│       └── NotifierPayload.kt
├── network/
│   ├── HttpClientFactory.kt
│   ├── RetrofitFactory.kt
│   └── JsonConfig.kt
├── audio/
│   └── SoundEffectPlayer.kt
├── database/
│   ├── CatsCarsDatabase.kt
│   ├── converters/
│   │   └── RoomConverters.kt
│   └── rawquery/
│       └── RawQueryRunner.kt
├── feature/
│   ├── splash/
│   │   ├── ui/
│   │   │   └── SplashScreen.kt
│   │   └── nav/
│   │       └── SplashEntry.kt
│   ├── main/
│   │   ├── ui/
│   │   │   └── MainScreen.kt
│   │   └── nav/
│   │       └── MainEntry.kt
│   ├── about/
│   │   ├── ui/
│   │   │   └── AboutScreen.kt
│   │   └── nav/
│   │       └── AboutEntry.kt
│   ├── cats/
│   │   ├── data/
│   │   │   ├── remote/
│   │   │   │   ├── CatApiService.kt
│   │   │   │   └── dto/
│   │   │   │       └── CatDto.kt
│   │   │   └── local/
│   │   │       └── CatEntity.kt
```

```

├── CatFtsEntity.kt
│   └── CatDao.kt
├── mapper/
│   └── CatMappers.kt
├── repository/
│   └── CatRepositoryImpl.kt
├── domain/
│   ├── model/
│   │   └── Cat.kt
│   ├── repository/
│   │   └── CatRepository.kt
│   └── usecase/
│       ├── GetCats.kt
│       ├── RefreshCats.kt
│       └── SearchCats.kt
├── ui/
│   ├── list/
│   │   ├── CatListScreen.kt
│   │   ├── CatListViewModel.kt
│   │   └── CatListState.kt
│   ├── detail/
│   │   ├── CatDetailScreen.kt
│   │   └── CatDetailViewModel.kt
│   └── component/
│       └── CatCard.kt
├── nav/
│   └── CatsEntry.kt
├── di/
│   └── CatModule.kt
├── vehicles/
│   ├── data/
│   │   ├── remote/
│   │   │   ├── VpicApiService.kt
│   │   │   ├── VpicEndpoint.kt
│   │   │   └── dto/
│   │   │       ├── VpicResponse.kt
│   │   │       ├── ManufacturerSummary.kt
│   │   │       ├── ManufacturerDetail.kt
│   │   │       ├── ManufacturerMake.kt
│   │   │       └── VehicleType.kt
│   │   └── local/
│   │       ├── ManufacturerEntity.kt
│   │       ├── ManufacturerDao.kt
│   │       ├── ManufacturerFtsEntity.kt
│   │       ├── MakeEntity.kt
│   │       ├── MakeDao.kt
│   │       └── MakeFtsEntity.kt
│   │   └── mapper/
│   │       └── VpicMappers.kt
│   └── repository/
│       └── VehicleRepositoryImpl.kt
├── domain/
│   ├── model/
│   │   ├── Manufacturer.kt
│   │   ├── ManufacturerDetail.kt
│   │   └── Make.kt
│   └── repository/

```

```

├── VehicleRepository.kt
├── usecase/
│   ├── GetManufacturers.kt
│   ├── GetManufacturerDetail.kt
│   ├── GetMakes.kt
│   └── SearchVpic.kt
├── ui/
│   ├── list/
│   │   ├── ManufacturerListScreen.kt
│   │   ├── ManufacturerListViewModel.kt
│   │   └── ManufacturerListState.kt
│   ├── detail/
│   │   ├── ManufacturerDetailScreen.kt
│   │   └── ManufacturerDetailViewModel.kt
│   ├── search/
│   │   ├── VpicSearchScreen.kt
│   │   └── VpicSearchViewModel.kt
│   └── component/
│       ├── ManufacturerCard.kt
│       └── MakeCard.kt
├── nav/
│   └── VehiclesEntry.kt
├── di/
│   └── VehicleModule.kt
├── coins/
│   ├── data/
│   │   ├── remote/
│   │   │   ├── CoinbaseApiService.kt
│   │   │   └── dto/
│   │   │       └── SpotPriceDto.kt
│   │   └── local/
│   │       ├── CoinEntity.kt
│   │       └── CoinDao.kt
│   ├── mapper/
│   │   └── CoinMappers.kt
│   ├── repository/
│   │   └── CoinRepositoryImpl.kt
│   ├── domain/
│   │   ├── model/
│   │   │   └── SpotPrice.kt
│   │   ├── repository/
│   │   │   └── CoinRepository.kt
│   │   └── usecase/
│   │       ├── ObserveSpotPrice.kt
│   │       └── ForcePoll.kt
│   ├── ui/
│   │   ├── CoinsScreen.kt
│   │   ├── CoinsViewModel.kt
│   │   └── CoinsState.kt
│   ├── nav/
│   │   └── CoinsEntry.kt
│   └── di/
│       └── CoinModule.kt
├── preferences/
│   ├── data/
│   │   ├── PreferencesDataSource.kt
│   │   └── UserPreferencesRepositoryImpl.kt

```

```

├── domain/
│   ├── model/
│   │   ├── UserPreferences.kt
│   │   └── ThemeMode.kt
│   ├── repository/
│   │   └── UserPreferencesRepository.kt
│   ├── usecase/
│   │   ├── ObservePreferences.kt
│   │   ├── SetTheme.kt
│   │   ├── SetPolling.kt
│   │   ├── SetPaused.kt
│   │   ├── SetHaptics.kt
│   │   └── ResetApp.kt
│   └── effect/
│       ├── ResetEffect.kt
│       └── SoundResetEffect.kt
├── ui/
│   ├── PreferencesScreen.kt
│   ├── PreferencesViewModel.kt
│   ├── PreferencesState.kt
│   └── component/
│       ├── ThemeSegmentedButton.kt
│       ├── PollingSlider.kt
│       ├── PauseSwitch.kt
│       ├── HapticsSwitch.kt
│       └── ResetButton.kt
├── nav/
│   └── PreferencesEntry.kt
└── di/
    └── PreferencesModule.kt
database/
├── data/
│   ├── TableRegistry.kt
│   └── DatabaseViewerRepositoryImpl.kt
├── domain/
│   ├── model/
│   │   ├── TableName.kt
│   │   ├── TableData.kt
│   │   └── SortState.kt
│   ├── repository/
│   │   └── DatabaseViewerRepository.kt
│   └── usecase/
│       ├── ObserveTable.kt
│       ├── DeleteRecord.kt
│       ├── RunSelect.kt
│       ├── RefreshTable.kt
│       ├── SearchTable.kt
│       └── CountRecords.kt
├── ui/
│   ├── DatabaseViewerScreen.kt
│   ├── DatabaseViewerViewModel.kt
│   ├── DatabaseViewerState.kt
│   └── component/
│       ├── TableSegmentedSelector.kt
│       ├── ControlBox.kt
│       ├── SortableHeaderRow.kt
│       └── ReadOnlyBadge.kt

```

- ```

├── nav/
│ ├── DatabaseEntry.kt
│ └── di/
│ └── DatabaseModule.kt
├── res/
│ ├── values/
│ │ ├── themes.xml
│ │ ├── colors.xml
│ │ └── strings.xml
│ ├── values-v31/
│ │ └── themes.xml
│ ├── drawable/
│ │ ├── splash_black.xml
│ │ └── splash_logo.xml
│ ├── mipmap-anydpi-v26/
│ │ ├── ic_launcher.xml
│ │ └── ic_launcher_round.xml
│ ├── raw/
│ │ └── db_wipe.ogg
├── test/
└── androidTest/

```

mipmap-anydpi-v26 icons; res/raw/db\_wipe.ogg.

## 0.5 App skeleton

- ☐ MyApp (Application) -> startKoin { modules(appModules) } (empty stub).
- ☐ MainActivity -> installSplashScreen(), setContent { AppTheme { NavSuite + NotifierHost(placeholder) + NavDisplay } }.
- ☐ AppTheme with light/dark palettes (dynamic off); Nav3 NavDisplay Splash -> Main -> empty stubs.

## 0.6 Verify

- ☐ ./gradlew assembleDebug green.
- ☐ One instrumented smoke test: launch MainActivity, assert Main renders; ./gradlew connectedDebugAndroidTest green.
- ☐ ./gradlew koverHtmlReport runs (baseline 0%).

## 0.7 Clean

- ☐ .gitignore covers local.properties, /build, IDE noise; schema-export dir tracked intentionally.
- ☐ Remove template starter cruft ( Greeting, sample previews).

## 24.1 Phase 1 — Preferences core (shared dependency)

- ☐ domain: UserPreferences, ThemeMode; repo interface; use cases (Observe/Set...).
- ☐ data: DataStore-backed repo. **Tests first** against a temp DataStore.
- ☐ (Screen deferred; Coins needs the repo, not the UI.)

## 24.2 Phase 2 — Coins (simplest feature; first full slice)

- ☐ domain: SpotPrice, ObserveSpotPrice (interval + dedup), ForcePoll. **Tests first** (Turbine + virtual time).
- ☐ data: Coinbase service; mapper (string USD -> amount\_cents); repo; CoinDao insert + trimToLimit. **Tests first.**
- ☐ ui: CoinsViewModel (pause/interval); CoinsScreen; wire Notifier toast on change.

## 24.3 Phase 3 — Cats (medium) · builds shared-infra #1

- ☐ SwipeableRefreshableList<T> — built + Compose-tested here.
- ☐ domain/data: GetCats / RefreshCats / SearchCats; repo (cache + network); CatDao + cats\_fts. **Tests first.**
- ☐ ui: list (swipe/refresh), detail, CatCard, Coil.

## 24.4 Phase 4 — Cars / VPIC (most complex)

- ☐ remote: VpicResponse<T>, endpoints, DTOs. **Mapper tests first** (envelope + nullable fields).
- ☐ data: paged manufacturer load (limit + spinner); lazy makes; \*\_fts population.
- ☐ ui: manufacturer list -> detail -> nested Makes (reuses the list); VPIC search screen; Nav3 adaptive panes.

## 24.5 Phase 5 — Database Viewer (capstone) · builds shared-infra #2

- ☐ SchemalessGrid (frozen header + shared h-scroll + header-button taps).
- ☐ data: TableRegistry; @RawQuery read/count/delete; per-segment refresh via each feature's RefreshableTable / ClearableTable.
- ☐ domain: ObserveTable / DeleteRecord / RunSelect / RefreshTable / SearchTable / CountRecords. **Guard tests first** (SELECT + first-; -chunk; read-only-if-not-plain-table).
- ☐ ui: segmented selector; 2x2 control box; tri-state sort; swipe-delete on plain views; read-only label + 750 ms toast; the in-app SQL popover.

## 24.6 TDD approach

### Test matrix

| Layer | What to test | Tools | Assert (examples) |
|-------|--------------|-------|-------------------|
|       |              |       |                   |

|                     |                             |                                                      |                                                                                                                      |
|---------------------|-----------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| domain (usecases)   | pure logic; interfaces      | JUnit4, coroutines-test, Turbine                     | dedup emits only on <code>amountCents</code> change; poll respects interval                                          |
| data (mappers)      | DTO <-> domain              | JUnit4 + JSON fixtures                               | VPIC envelope decodes; <code>"63142.55"</code> -> <code>6314255</code>                                               |
| data (repos)        | cache/network orchestration | hand-written fakes; MockK sparingly                  | refresh restores removed items; cache-then-network                                                                   |
| data (Room DAO/FTS) | queries, FTS, FIFO          | Room-testing, instrumented / Robolectric             | <code>cats_fts</code> matches; <code>trimToLimit</code> evicts lowest <code>id</code> ; <code>DELETE WHERE id</code> |
| ui (ViewModel)      | state transitions           | coroutines-test, <code>StandardTestDispatcher</code> | pause freezes; interval change restarts flow                                                                         |
| ui (Compose)        | behavior, not pixels        | <code>createComposeRule</code>                       | swipe removes; pull-refresh restores; header tap toggles sort                                                        |

## Principles (in priority order)

1. **Domain-first.** Write domain tests before data/ui — pure Kotlin, fastest, and the tests drive the interfaces (DIP falls out of test order).
2. **Fakes over mocks.** Hand-written fakes for repositories/APIs; reserve MockK for the rare case a fake is impractical. Refactor-resilient.
3. **Inject dispatchers.** No hard-coded `Dispatchers.IO`; pass a `DispatcherProvider` so `StandardTestDispatcher` controls time.
4. **Deterministic Flows.** Test Flow with Turbine + `runTest` virtual time — no real delays.
5. **Coverage via Kover.** High target on domain/data/viewmodels; pragmatic on composables (no 100%-on-UI chase).

**Shared-infra build points:** #1 generic list (Phase 3); #2 `SchemalessGrid` (Phase 5); `Notifier` first built Phase 2, reused Phase 5.